Software Architecture - Group Assignment 02

Architectural Evolution of nopCommerce: Federated Commerce After Acquisitions

Part 2 - Implementation, Evidence, and Defense

Ricardo Dias [108598], Sérgio Correia [108647]

v2026-05-31

Abstract

This report is Part 2 of the Northstar architectural evolution, documenting the running counterpart of the design proposed in Part 1. The deep current-state analysis, the seven pressure points, the full bounded-context derivation, the six-part Quality Attribute scenarios, the Attribute-Driven Design iteration log, and the rejected alternatives for each Architecture Decision Record already live in the Part 1 report and are not duplicated here. What Part 2 adds is the implementation of that design as concrete plugins and workers, and the five ADRs revisited in their Accepted form: each ADR lives as a self-contained subsection that combines implementation, verification with reproduction script and dashboard panel, trade-off, and any gap between the design and the code together with its production fix. The scenario's pressure point, a business-unit-local subsystem degrading without taking the storefront or the group down, is realised by a circuit breaker around the BU-local ERP integration and is demonstrated end to end by a script that produces a clean state-machine trace in the dashboard.

Contents

1. Introduction

Part 1 of this assignment produced an architectural evolution of nopCommerce for Scenario A and concluded with five Architecture Decision Records in Proposed status. Part 2 is the implementation of that design and the evidence that it works under pressure.

This report is deliberately short. Anything that Part 1 already establishes is cited by Part 1 section rather than re-rendered:

- the scenario;
- the current-state analysis;
- the bounded-context map;
- the QA scenarios in full six-part form;
- the framework choice;
- the rejected alternatives per ADR;
- the runtime sequence diagrams for SSO, order-to-CRM, and ERP failure.

Part 2 contains only what is genuinely new in the implementation phase.

1.1 Document structure

Section 2 describes what was built. Section 3 covers the five Architecture Decision Records, each self-contained: implementation, verification with reproduction script and dashboard panel, trade-off, and limitation. Section 4 names the operational concerns the demo deliberately does not exercise. Section 5 is the reproduce-the-demo guide.

2. What Was Built

2.1 Topology

Two nopCommerce processes run side by side against two private PostgreSQL databases. Both BUs use the same container image, parameterised by environment variables. Neither BU has any path into the other's database, App_Data volume, or settings table.

Around the two storefronts sit:

- **Keycloak** as the group identity provider, with one OIDC client per BU;
- **RabbitMQ** as the durable broker for cross-BU events, with a topic exchange and a dead-letter queue;
- **Meilisearch** as the federated search index, with a per-BU filter attribute;
- **EspoCRM** as the customer source of truth, fed by a dedicated consumer worker;
- **ErpStub** containers, one per BU from a single image, with toggleable break/recover endpoints;
- an **nginx portal** serving the cross-BU landing page.

2.2 First-party code

Four nopCommerce plugins implement the four seams; the design rationale is in Part 1:

- `ExternalAuth.Keycloak`: the per-BU OIDC handler;
- `Misc.OutboxRelay`: the `OrderPlacedEvent` hook plus the background-service relay that ships outbox rows to the broker;
- `Misc.ErpIntegration`: the circuit breaker around stock lookups and the "stock to be confirmed" banner;
- `Search.Meilisearch`: the `ISearchProvider` implementation with a 1.5-second timeout and a fallback to the vendor SQL search.

Two standalone workers run outside the nopCommerce processes: `ErpStub`, described above, and `EspoCrmConsumer`, which is independently deployable, subscribes to the broker, and upserts EspoCRM contacts.

The only edit to vendor code is three minimally invasive patches to `Nop.Services.Catalog.ProductService`, needed because its expression tree does not translate when a search provider supplies a candidate ID list. Every other change is additive and lives in a plugin or worker.

2.3 Observability spine

The observability overlay `docker-compose.observability.yml` adds Prometheus and Grafana without touching the main compose. Prometheus scrapes both nopCommerce processes through an exporter wired once in `Nop.Web/Program.cs`, but deliberately not from the storefront port: a `MapWhen` on a dedicated metrics port (9101) branches `/metrics` onto a minimal middleware chain that never runs the DB-bound nopCommerce pipeline, so a BU stays scrapable even while its own database is down (this is what lets the ADR-001 isolation demo keep producing evidence through the outage rather than going dark with the storefront). It reaches RabbitMQ, Meilisearch, the EspoCRM consumer, and Keycloak

through their native Prometheus integrations. On top of those infrastructure metrics, the three failure-mode plugins emit their own OpenTelemetry meters: a breaker-state gauge, a search-fallback counter and duration histogram, and an outbox publish counter and latency histogram. A k6 load harness drives request load from inside the same Docker network, running search, checkout, and product-detail scenarios and remote-writing the results to Prometheus so the dashboard sees production-shaped traffic during the demos. All of these signals surface through a single Grafana dashboard, provisioned automatically with one row per failure-mode ADR.

3. Accepted Architectural Decisions

All five ADRs from Part 1 move from Proposed to Accepted. The Context, Decision, and Rejected alternatives prose for each ADR is in Part 1 (Section 7) and is not duplicated here. Each subsection below is self-contained: implementation, verification with reproduction script and dashboard panel, trade-off, and limitation.

3.1 ADR-001: Per-BU process and database isolation

Accepted 2026-05-30. Drivers: QA1, QA2. Full design rationale: Part 1 ADR-001.

Implementation. The isolation is structural rather than configurational. The two BUs run as separate nopCommerce processes, one per business unit, each against its own private PostgreSQL database. Both processes use the same container image; the differences between them are purely environment-driven through the BU identifier, the connection string, the per-BU OIDC client credentials, and the per-BU ERP URI. The installer wires the right values into each BU's `App_Data` at first-run installation.

Nothing crosses the boundary in-process. No connection string, settings table, App_Data volume, or in-memory state is shared between the two BUs, so a memory leak, a slow plug-in, a runaway query, or a database connection-pool exhaustion in one BU cannot starve the other. The group-level systems Keycloak, RabbitMQ, Meilisearch, and EspoCRM are reached over the network and never through shared process state.

Element	Where
Per-BU containers	`docker-compose.yml`: `nop_bu1`, `nop_bu2`
Per-BU PostgreSQL	`docker-compose.yml`: `db_bu1`, `db_bu2`
App_Data volumes	`nop_bu1_app_data`, `nop_bu2_app_data`
Seed scripts	`scripts/seed-bu1.sql`, `scripts/seed-bu2.sql`
Smoke test	`scripts/test-isolation.sh`

Verification. The smoke test `scripts/test-isolation.sh` performs three steps:

1. `docker stop northstar-db_bu1-1`
2. `curl -fsS http://localhost:8082/` expects HTTP 200
3. `docker start northstar-db_bu1-1`

The isolation proof is the smoke test: BU2 keeps answering HTTP 200 for the full 150-second window during which `db_bu1` is stopped. To quantify the storefront's load envelope, k6 product-detail-load drives the storefront at up to 30 virtual users for three minutes.

Observed:

- BU2 returned HTTP 200 on all 75 probes throughout the outage (0 failures). That is the structural proof that BU1's DB outage did not propagate.
- The k6 load baseline sustained 2,764 requests at 0.00% errors with p95 491 ms under 30 VUs, comfortably inside the `http_req_failed < 0.05` threshold.

```
$ ./scripts/test-isolation.sh
==> Generating baseline traffic on both BUs (10s)...
```

```
==> Stopping db_bu1...
 Container northstar-db_bu1-1  Stopped
==> db_bu1 is DOWN for 150s.

==> Launching BU1 background requests (waiting for 500 responses)...
BU1 background request 1: HTTP 500
BU1 background request 2: HTTP 500
BU1 background request 3: HTTP 500
... (10 requests, all HTTP 500)

==> Polling BU2 every 2s to prove isolation...
BU2=200 (ok=1 fail=0)
BU2=200 (ok=2 fail=0)
BU2=200 (ok=3 fail=0)
... (75 polls, all HTTP 200)

==> RESULT:
         BU2 (isolated): 75 ok, 0 fail
PASS: BU2 was 100% available throughout the BU1 DB outage.

==> Restoring db_bu1...
 Container northstar-db_bu1-1  Started
==> db_bu1 restarted. Waiting 15s for BU1 to recover...
==> BU1 after recovery: HTTP 200

$ ./scripts/run-loadtest.sh product-detail-load   # storefront load baseline
     THRESHOLDS
       http_req_duration{scenario:product_detail}  p(95)=491.48ms   ok 'p(95)<3000'
       http_req_failed{scenario:product_detail}     rate=0.00%       ok 'rate<0.01'
     TOTAL RESULTS
       checks_succeeded ...: 100.00% (2764 out of 2764)   status 200
       http_req_duration ..: avg=208.66ms  med=165.71ms  p(90)=389.07ms  p(95)=491.48ms  max=1.16s
       http_req_failed ....: 0.00%   (0 out of 2764)
       http_reqs ..........: 2764    15.34/s
       vus_max ............: 30      (ramp 0->15->30->0 over 3m)
```

Trade-off. Doubled infrastructure and operational surface, accepted because fault isolation is now structural rather than configurational.

Limitation. None.

3.2 ADR-002: Keycloak group SSO via OIDC

Accepted 2026-05-30. Driver: QA3. Full design rationale: Part 1 ADR-002.

Implementation. Keycloak hosts the group realm with one OIDC client per BU. nopCommerce sits as a relying party through a custom plugin, `Nop.Plugin.ExternalAuth.Keycloak`, which implements `IExternalAuthenticationMethod` and registers `AddOpenIdConnect` once per BU with that BU's client credentials.

The non-obvious detail in the wiring is the OIDC response mode. The default is `form_post`, which delivers the authorisation code back to the relying party via a cross-site POST. With SameSite=Lax cookies, the browser drops the correlation cookie on that POST and the second-leg login silently fails. The plugin therefore forces `ResponseMode = "query"` so the code comes back on a same-site GET, where the cookie survives the redirect.

Two smaller details round out the integration. The plugin is built with `Microsoft.NET.Sdk.Web` rather than the default SDK because the OIDC handler lives in the ASP.NET Core shared framework and is not redistributable as a regular

NuGet. A host alias for the Keycloak hostname is pinned via `extra_hosts` so the browser-side hostname matches the OIDC discovery document, and the realm-import file is mounted into the Keycloak container at boot, so the two OIDC clients and the test user exist on first run without any manual setup.

Per-BU role isolation is enforced at the token boundary. Each client carries a per-client role mapper (`oidc-usermodel-client-role-mapper`) that emits a `bu_roles` claim containing *only that client's* roles. Test user `alice` holds a role on both clients (`HomeStyle-VIP` on BU1, `WorkSpace-Wholesale` on BU2), yet a token minted for one client cannot contain the other client's roles: the isolation is structural, not a post-hoc filter. On `OnTokenValidated` the plugin maps `bu_roles` into local roles prefixed with the BU identifier (`${BU_ID}:{role}`), matching the Part 1 cross-cutting concern that local role IDs are BU-prefixed to prevent cross-BU collision.

Element	**Where**
Realm definition	`spike/keycloak/realm-northstar.json`, mounted into Keycloak at boot
Plugin	`src/Plugins/Nop.Plugin.ExternalAuth.Keycloak/`
Per-BU env vars	`docker-compose.yml`: `KEYCLOAK_AUTHORITY`, `KEYCLOAK_CLIENT_ID`, `KEYCLOAK_CLIENT_SECRET`
Hostname pinning	`extra_hosts: ["keycloak.localtest.me:host-gateway"]` so the browser-side hostname matches the discovery doc
Smoke test	`scripts/test-sso.sh`

Verification. The smoke test `scripts/test-sso.sh` exercises the full SSO loop with `curl` and a cookie jar:

1. Performs the initial login against BU1, captures the session cookie and ID token.
2. Hits BU2's `/login` endpoint reusing the Keycloak browser session.
3. Compares the resulting BU2 response to the assertion.

Expected:

- The second-leg redirect completes in under two seconds.
- No credential prompt is shown.
- Each BU's token carries only its own role: the BU2 token has `WorkSpace-Wholesale` and not `HomeStyle-VIP`, and vice versa (zero role bleed-through, even though `alice` holds both roles in Keycloak).

```
$ ./scripts/test-sso.sh
==> 1. Fetch Keycloak login page for bu1-nopcommerce
    form action: http://keycloak.localtest.me:8080/realms/northstar/login-...
==> 2. POST credentials for alice
    OK - Keycloak issued a code for BU1
==> 3. Re-authorize for bu2-nopcommerce using the same cookie jar (SSO probe)
    OK - Keycloak issued a code for BU2 without re-prompt

SSO VERIFIED: single login -> tokens for BU1 and BU2
==> 4. Role isolation: assert zero role bleed-through across BUs
    BU1 token bu_roles: HomeStyle-VIP
    BU2 token bu_roles: WorkSpace-Wholesale
    OK - each BU token carries only its own role (alice holds both in Keycloak)

ROLE ISOLATION VERIFIED: zero role bleed-through (QA3)
```

Trade-off. Keycloak is a hard dependency at login time, but storefront browsing does not hit it.

Limitation. One demo-vs-production gap remains.

The realm import file is a development-only Keycloak realm. In production the realm would be migrated via Keycloak's REST API as part of a deploy pipeline rather than being mounted from a JSON file at boot. (Per-BU role mapping, previously deferred, is now implemented and verified above.)

3.3 ADR-003: Transactional outbox to RabbitMQ to EspoCRM

Accepted 2026-05-30. Driver: QA5. Full design rationale: Part 1 ADR-003.

Implementation. The pipeline has three stages. When an order is placed in any BU, the in-process `OrderPlacedConsumer` hooks the nopCommerce `OrderPlacedEvent` and writes an `OutboxMessage` into that BU's own database via `OutboxWriter`. The row carries the BU identifier, the event type, the JSON payload, and a `PublishedAt` timestamp left null until the relay ships it.

`OutboxRelayService`, a hosted background service running inside each nopCommerce process, polls unpublished outbox rows on a short interval, publishes them to the broker on the `northstar.events` topic exchange with routing key `{buId}.order.placed`, and then sets `PublishedAt` on the rows it successfully published. Failures stay in the table for the next poll round, and a dead-letter queue, `northstar.dlx`, catches messages that cannot be delivered after the retry budget.

The `EspoCrmConsumer` worker is an independently deployable .NET background service running outside the storefront processes. It subscribes to the `northstar.crm.orders` queue and translates each incoming event into a REST upsert against the EspoCRM API. The upsert is idempotent on `OrderId`, so a duplicate delivery from the broker does not produce a duplicate contact.

Contact matching is keyed on the customer's email: for each order the consumer looks up the EspoCRM contact by `emailAddress` and, if one already exists, accumulates the new purchase onto that shared contact instead of creating a second one. A customer who buys from both BUs therefore lands on a single unified contact carrying the cross-BU purchase history. That view is assembled only from the order events each BU publishes, so neither BU ever reads the other's database and the ADR-001 isolation boundary is preserved.

Element	Where
Outbox entity + migrator	`src/Plugins/Nop.Plugin.Misc.OutboxRelay/Domain/`, `Data/`
Order hook	`Consumers/OrderPlacedConsumer.cs`
Outbox writer	`Services/OutboxWriter.cs`
Relay background service	`Services/OutboxRelayService.cs`
RabbitMQ publisher	`Services/RabbitMqPublisher.cs`
Consumer worker	`src/Workers/EspoCrmConsumer/`
Broker topology	exchange `northstar.events` (topic, durable); queue `northstar.crm.orders` durable, persistent delivery (`delivery_mode=2`), no message TTL; DLQ `northstar.dlx`
Smoke + demo	`scripts/test-outbox.sh`, `scripts/demo-outbox.sh`, `scripts/test-durability.sh`, `scripts/demo-cross-bu-crm.sh`

Verification. The smoke test `scripts/test-outbox.sh` places a synthetic order in BU2 and polls EspoCRM.

The durability demo `scripts/demo-outbox.sh` pauses the EspoCRM consumer, injects 100 synthetic `OutboxMessage` rows into `db_bu1` via `psql`, watches the queue depth climb, then unpauses.

The QA5 measure "the message survives at least 24h of consumer downtime" is a property of the broker configuration, not a wall-clock wait: the queue is durable, messages are persisted (`delivery_mode=2`), and no message TTL is set, so the survival window is bounded only by broker disk. `scripts/test-durability.sh` proves the mechanism that backs this SLA, which is strictly stronger than a timed pause: it stops the consumer, publishes N orders, restarts RabbitMQ to force on-disk persistence, confirms the queue is intact after the broker bounce, then drains with zero loss.

The cross-BU customer demo `scripts/demo-cross-bu-crm.sh` places one order from BU1 and one from BU2 for the same customer email, then reads the resulting EspoCRM contact back to show both purchases folded onto a single profile. It is the end-to-end proof of the shared-customer use case from Scenario A.

Expected:

- Smoke path: the corresponding CRM contact appears in EspoCRM well within the 30 s SLA (the test reports the measured time).
- Durability path: the queue depth climbs to around one hundred during the pause and drains to zero after unpause.
- The EspoCRM contact count rises by exactly one hundred, proving at-least-once delivery plus idempotent consumption.
- Broker-restart path: all N messages persist across a full RabbitMQ restart and are processed with zero loss once the consumer returns.
- Cross-BU view: a customer buying from both BUs yields exactly one EspoCRM contact whose history lists both the BU1 and the BU2 order.

```
$ ./scripts/test-outbox.sh
==> 1. Checking espocrm_consumer container...
    espocrm_consumer: running
==> 2. Publishing synthetic order to northstar.events (key: bu2.order.placed)...
    OK - message published and routed
==> 3. Waiting up to 30s for EspoCRM contact to appear...
```

```
PASS: EspoCRM contact for outbox-test-...@northstar.test created in 3s (within the 30s QA5 SLA).

$ ./scripts/demo-outbox.sh          # durability through a CRM consumer outage
[Pausing espocrm_consumer - relay publishes but no one consumes]
[Inserting 100 synthetic outbox rows on nop_bu1]
  rabbitmq_queue_messages{northstar.crm.orders} = 100   (expected ~100)
[Holding the outage 60s - queue depth plateaus]
  queue depth = 100 ... 100 (12 samples, steady)
[Unpausing espocrm_consumer - queue drains]
  queue depth = 100 -> 80 -> 59 -> 38 -> 17 -> 0
  queue depth = 0 (steady)
[Done. queue climbed to ~100, then drained to 0 - no messages lost]

$ ./scripts/test-durability.sh 20     # survives consumer downtime + broker restart
==> 1. Stopping espocrm_consumer (simulate consumer downtime)
==> 2. Publishing 20 synthetic orders to northstar.events
==> 3. Queue depth before broker restart: 20 (expected 20)
==> 4. Restarting RabbitMQ (proves on-disk persistence)...
==> 5. Queue depth AFTER broker restart: 20 (expected 20)
    OK - all 20 messages persisted across the broker bounce
==> 6. Starting espocrm_consumer; waiting for drain + CRM upsert
    all 20 contacts present after 6s
PASS: 20/20 orders survived consumer downtime + a broker restart with zero loss (QA5).

$ ./scripts/demo-cross-bu-crm.sh    # same customer buys in BOTH BUs
Customer: alice-...@northstar.test  (same identity in both BUs)
  OK - bu1 order #119388 (149.99) published and routed
  OK - bu2 order #111467 (299.00) published and routed
[waiting for the consumer to fold both orders into one contact]
  contacts matching this email : 1   (expected 1 - one customer, not one-per-BU)
  cross-BU purchase history:
      [bu1] Order #119388 | Total: 149.99
      [bu2] Order #111467 | Total: 299.00
PASS: orders from BU1 and BU2 both visible on ONE shared customer profile
      (no BU read the other's database - built only from published events).
```

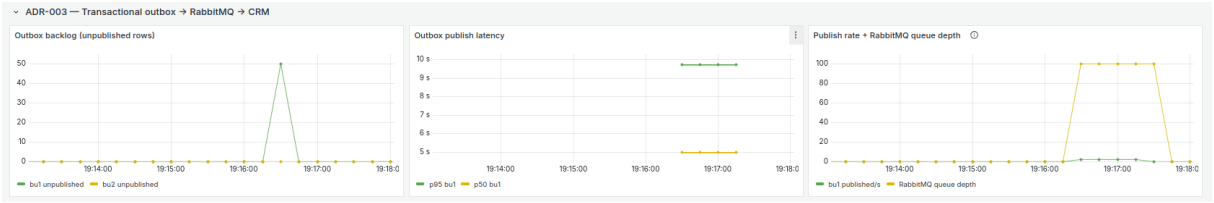


Figure 1: ADR-003 row captured during `demo-outbox.sh`. Left: outbox backlog peaks at 50 unpublished rows while the consumer is paused. Right: RabbitMQ queue depth climbs to 100 then drains to 0 within five seconds of unpause, confirming at-least-once delivery with no message loss during a consumer outage.

Trade-off. Visibility lag of seconds rather than instant CRM updates, accepted to avoid coupling checkout to CRM availability.

Limitations. Three gaps remain between the ADR text in Part 1 and the code.

First, the outbox insert is in a separate transaction from the order insert. Part 1's text claims that the order row and the outbox row are written in the same database transaction, but in practice `PlaceOrderAsync` does not wrap the entire flow in `BeginTransactionAsync`: the order commits first and the `OrderPlacedEvent` fires after, so the `OutboxMessage` insert is a second transaction. The window during which the order exists but the outbox row does not is sub-millisecond, mitigated by the container's `restart: on-failure` policy. The production fix is either a vendor patch that wraps `PlaceOrderAsync` in an outer transaction, or a database trigger on the `Order` table that writes the outbox row.

Second, the consumer's idempotency is in-memory only. `EspoCrmConsumer` keeps a `HashSet<string>` of processed `OrderId`s, which survives in-process duplicate delivery but resets when the container restarts. If the consumer crashes during recovery and restarts while messages are still pending, it could produce duplicate EspoCRM contacts. The production fix is to persist the processed-event keys in EspoCRM via a custom field, or in a small dedicated table.

Third, the `OrderMessage` payload has no explicit `version` field, contrary to the cross-cutting concerns in Part 1. None of the current consumers branches on version so the field was not added; the production fix is trivial.

3.4 ADR-004: Circuit breaker around BU-local ERP

Accepted 2026-05-30. Driver: QA1. Full design rationale: Part 1 ADR-004.

Implementation. The plugin protects the BU-local ERP call with two layers: an in-process cache and a circuit breaker. `ErpStockService` is the HTTP entry point. It first checks an `IMemoryCache` keyed on `erp.stock.{sku}` with a five-minute TTL. On a cache miss it goes through `ErpCircuitBreaker`, which wraps the live HTTP call.

The breaker has three states. CLOSED is the healthy path: requests pass through and live results flow back. After five consecutive failures the breaker moves to OPEN, where it fast-fails every call without touching the ERP for a 30-second cooldown. After the cooldown it moves to HALF-OPEN and allows a single probe through; a successful probe closes the breaker back to CLOSED, a failed probe reopens it for another cooldown.

When the breaker is OPEN, `ErpStockService` short-circuits on the breaker's `IsOpen` flag and returns the last cached stock value with `IsStale = true`. `ErpStockBannerViewComponent` reads that flag and renders the "stock to be confirmed" banner on the product page. The page itself never returns a 5xx, so the customer sees a labelled degraded experience rather than an outage.

The ERPs themselves are the `ErpStub` worker, a single image used as both `erp_bu1` and `erp_bu2`. The stub exposes `POST /admin/break` and `POST /admin/recover` endpoints so the failure mode is deterministic and demoable end to end.

Element	Where
Plugin	`src/Plugins/Nop.Plugin.Misc.ErpIntegration/`
HTTP client	`Services/ErpStockService.cs`
State machine	`Services/ErpCircuitBreaker.cs`
Cache	`IMemoryCache` keyed `erp.stock.{sku}`, TTL 5 min
Banner UI	`ErpStockBannerViewComponent` (renders when `IsStale`)
ERP stub	`src/Workers/ErpStub/` (one image, two containers, `/admin/break` and `/admin/recover` toggles)
OpenTelemetry meters	`Services/ErpMetrics.cs` (state gauge 0/1/2, transitions counter, duration histogram)
Smoke + demo	`scripts/test-erp-failure.sh`, `test-erp-recovery.sh`, `demo-breaker.sh`

Verification. Three scripts cover failure, recovery, and the metrics shape.

The failure-path smoke test `scripts/test-erp-failure.sh` POSTs to `/admin/break` on `erp_bu2` and hits BU2 and BU1 product pages.

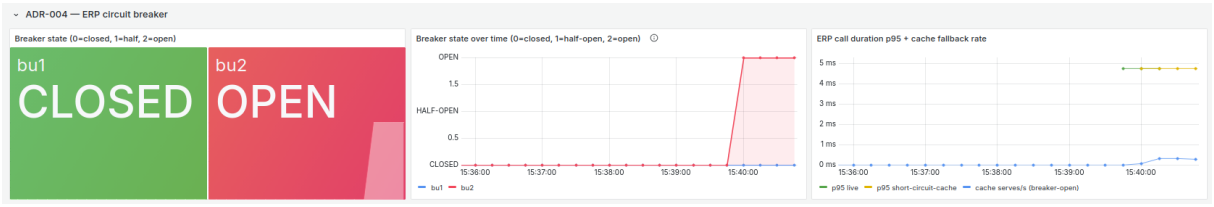
The recovery-path smoke test `scripts/test-erp-recovery.sh` POSTs to `/admin/recover` and waits for the half-open probe to close the breaker.

The metrics demo `scripts/demo-breaker.sh` starts k6 product-detail-load against BU1, breaks the BU1 ERP at T+40 s, and recovers at T+120 s.

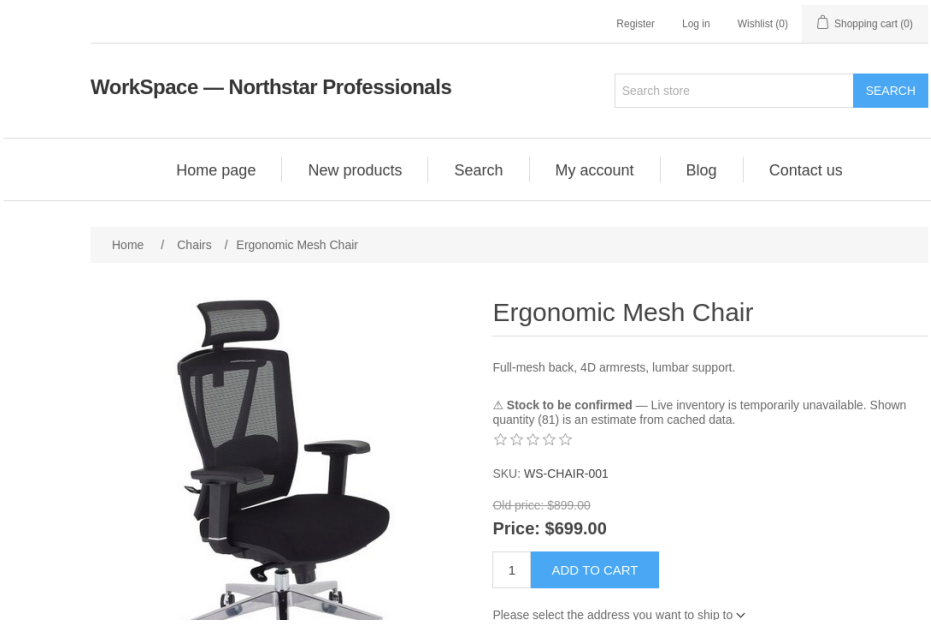
Expected:

- During failure: BU2 shows the "stock to be confirmed" banner and BU1 shows no banner. That is the isolation claim.
- After 30 to 60 seconds of recovery probe: the banner is gone on BU2.
- In the demo run: the ADR-004 dashboard row shows the state-gauge stepping $0 \rightarrow 2 \rightarrow 1 \rightarrow 0$, the transitions counter spiking at each change, and the call-duration histogram switching from the `live` outcome to `cache` during the open window. That trace is the literal proof the state machine executed.

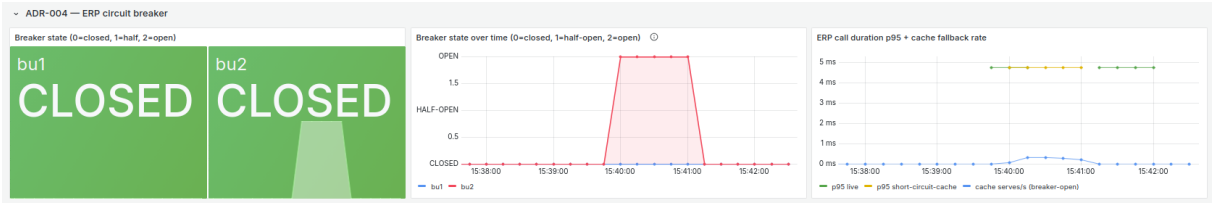
```
$ ./scripts/test-erp-failure.sh
1. Both ERPs healthy ..... bu1: ok   bu2: ok
2. Live stock from BU2 ERP ..... {sku: DEMO-SKU-001, quantity: 4, WH-BU2}
3. Breaking BU2 ERP ..... {status: broken, buId: bu2}
4. BU2 ERP /stock status ..... 503
5. BU1 ERP /stock status ..... 200   (BU1 is unaffected)
6. BU2 product pages during outage:
   Request 1..6: HTTP 200          (storefront serves cached stock, breaker OPEN)

$ ./scripts/test-erp-recovery.sh
1. Recovering BU2 ERP ..... {status: recovered, buId: bu2}
2. BU2 ERP responding normally ... {sku: DEMO-SKU-001, quantity: 4, WH-BU2}
3. Waiting 32s for breaker cooldown (OPEN -> HALF-OPEN -> CLOSED on probe)
=== Recovery demo complete ===

$ ./scripts/demo-breaker.sh         # k6 product-detail-load, break ERP mid-run
[t+40s] Breaking erp_bu1   -> breaker OPENs after 5 failed calls
[t+120s] Recovering erp_bu1 -> breaker probes HALF-OPEN, then CLOSES
     THRESHOLDS
       http_req_failed{scenario:product_detail}  rate=0.00%      ok 'rate<0.01'
       http_req_duration{...}                    p(95)=293.51ms  ok 'p(95)<3000'
     TOTAL RESULTS
       checks_succeeded ...: 100.00% (2936 out of 2936)   status 200
       http_req_failed ....: 0.00%   (0 out of 2936)
       http_reqs ..........: 2936    16.30/s
   Storefront served 200 on every request despite the ERP outage mid-run.
```



Grafana - circuit OPEN (state = 2)



BU2 storefront - stale banner



Grafana - circuit CLOSED (state = 0)

Figure 2: ADR-004 evidence. Top: Grafana confirms BU2 circuit breaker transitions to OPEN after five consecutive ERP failures. Middle: BU2 product page remains available with a labelled degraded banner instead of a 5xx - the storefront never goes down. Bottom: after ERP recovery the circuit closes automatically and normal stock figures resume.

Trade-off. Stale stock during the open window, accepted because the page never returns a 5xx.

Limitations. Three demo-vs-production gaps remain.

First, the 5-failure / 30-second cooldown thresholds are demo-tuned, not capacity-tuned. They would be adjusted in production based on the ERP's actual latency distribution and the storefront's traffic profile.

Second, the cache TTL of 5 minutes is generous for stock. Production would want SKU-

tiered TTLs and reactive invalidation on `OrderPlacedEvent` so the cached value tightens as the SKU sells through.

Third, there is no load shedding when the breaker is OPEN. Requests still arrive at the breaker and fast-fail there; for very high traffic, an upstream throttle would reduce wasted work.

3.5 ADR-005: Federated Meilisearch with SQL fallback

Accepted 2026-05-30. Driver: QA4. Full design rationale: Part 1 ADR-005.

Implementation. The search provider sits behind the standard nopCommerce `ISearchProvider` contract, so the storefront integration code stays unchanged. `MeilisearchSearchProvider` handles each call directly, with no separate strategy object: it tries Meilisearch first and lets nopCommerce's `ProductService` take over the fallback on failure.

The primary path issues the query against Meilisearch with a 1.5-second timeout. On success, the matched product IDs are returned. On the 1.5-second timeout or any other query exception (caught generically), the provider falls through to the fallback path: the vendor SQL search inside `ProductService`. An `IFallbackSignal` sets a ViewData flag that `SearchFallbackBannerViewComponent` reads to render the "limited results" banner to the customer.

The Meilisearch index uses a composite `{buId}-{productId}` document identifier and a filterable `buId` attribute, so the shared index can be queried per BU without cross-tenant leakage. The index stays in sync via `ProductSavedConsumer`, which hooks the vendor's product insert, update, and delete events and upserts or removes the corresponding document.

Three minimally invasive patches in vendor `Nop.Services.Catalog.ProductService` make this work. They gate the SKU, category, manufacturer, and tag union blocks behind a `runStandardSearch` flag so they are skipped when a provider already supplied a candidate ID list; they replace a query join with a materialised `Contains` filter; and they replace the provider-ordering join with an in-memory dictionary lookup that preserves the original sort semantics. These are the only vendor edits in the repository.

Element	**Where**
Plugin	`src/Plugins/Nop.Plugin.Search.Meilisearch/`
Search provider	`MeilisearchSearchProvider` (implements `ISearchProvider`)
Fallback strategy	`MeilisearchSearchProvider.cs` (1.5 s timeout, `FallbackSignal`)
Indexer	`Services/MeilisearchClient.cs`, `Services/MeilisearchIndexer.cs`
Index sync hook	`Consumers/ProductSavedConsumer.cs`
Recovery resync	`Services/MeilisearchResyncService.cs` (health probe, bulk reindex on recovery)
Banner UI	`SearchFallbackBannerViewComponent`
Vendor patches	`src/Libraries/Nop.Services/Catalog/ProductService.cs`
OpenTelemetry meters	`Services/SearchMetrics.cs`
Smoke + demo	`scripts/test-search.sh`, `scripts/demo-search-fallback.sh`

Verification. The smoke test `scripts/test-search.sh` exercises four flows: live, fallback, recovery, and post-recovery resync (it confirms both BUs' documents are back in the index within the QA4 budget after Meilisearch returns).

The metrics demo `scripts/demo-search-fallback.sh` runs k6 search-load while Meilisearch is paused at T+40 s and unpaused at T+90 s.

Expected:

- The "limited results" banner is present only during the fallback flow and absent in live and recovery.
- The ADR-005 dashboard row shows the `backend="meili"` latency line dropping out, the `backend="db"` line appearing with materially higher latency, and the fallback-rate counter spiking.
- The k6 panel shows no 5xx series at any point: the outage is invisible to the customer in terms of HTTP success.

```
$ ./scripts/test-search.sh
1. Meilisearch /health ........... {status: available}
3. Live search path (Meilisearch up)
    BU1: HTTP 200, fallback-banner=no
    BU2: HTTP 200, fallback-banner=no
4. Stopping meilisearch container to force fallback...
5. Fallback path (Meilisearch down)
    BU1: HTTP 200, fallback-banner=yes   (DB fallback in ProductService)
    BU2: HTTP 200, fallback-banner=yes
6. Restarting meilisearch...
7. Recovery path (banner should be gone)
    BU1: HTTP 200, fallback-banner=no
    BU2: HTTP 200, fallback-banner=no
8. Auto-resync after recovery (QA4: resync within 5 min)
    nop_bu1: resync completed after ~32s (log: 'Meilisearch resync complete')
    nop_bu2: resync completed after ~33s (log: 'Meilisearch resync complete')
    bu1: estimatedTotalHits=... after recovery
    bu2: estimatedTotalHits=... after recovery
    OK - resync fired and index is healthy, within the 5-minute QA4 budget

$ ./scripts/demo-search-fallback.sh   # k6 search-load, pause Meilisearch mid-run
[t+40s] Pausing Meilisearch  -> queries take the DB fallback path
[t+90s] Unpausing Meilisearch -> meili backend resumes
     THRESHOLDS
       http_req_failed{scenario:search}    rate=0.00%     ok 'rate<0.01'
       http_req_duration{scenario:search}  p(95)=3.09s    ok 'p(95)<5000'
     TOTAL RESULTS
       checks_succeeded ...: 100.00% (2361 out of 2361)   status 200
       http_req_failed ....: 0.00%   (0 out of 2361)
       http_reqs ..........: 2361    15.70/s
   Search stayed 200 across the full Meilisearch outage via DB fallback.
```

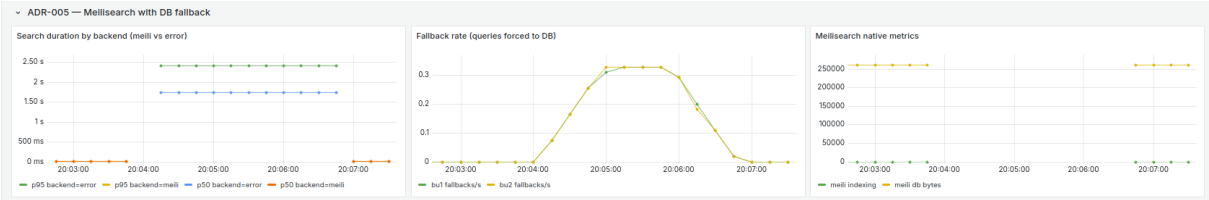


Figure 3: ADR-005 evidence captured during `demo-search-fallback.sh`. Left: search duration spikes to ∼2.5 s on the error backend while Meilisearch is paused, compared to ∼1.6 s on the live path. Center: fallback rate climbs to ∼0.3 queries/s across both BUs during the outage window, then drops to zero on recovery. Right: Meilisearch native metrics confirm the outage and resumption of the indexing pipeline.

Trade-off. Fallback latency is seconds rather than milliseconds, signalled to the customer by a "limited results" banner.

Limitations. Two gaps remain.

First, the fallback is timeout-driven rather than a formal circuit breaker. Part 1 describes a breaker with a half-open probe at thirty-second intervals, but the implementation uses a per-request 1.5-second timeout instead, with no CLOSED / HALF-OPEN / OPEN state machine. During a Meilisearch outage every search request pays the 1.5-second timeout before falling back, whereas a real breaker would short-circuit subsequent requests in the cooldown window. Recovery is still automatic. The production fix is to wrap the Meilisearch call in a Polly `CircuitBreakerPolicy` identical in shape to `ErpCircuitBreaker`.

Second, the vendor patches in `ProductService.cs` are upgrade-fragile and would need re-application or revisiting on any nopCommerce upgrade.

The recovery resync, previously deferred, is now implemented: `MeilisearchResyncService` (a hosted background service in the plugin) probes Meilisearch health every 30 seconds and, on a down $\rightarrow$ up transition, bulk-reindexes this BU's catalog. Detection plus reindex sit well inside the QA4 "resync within 5 minutes" budget, so product changes that occurred during an outage are reconciled on recovery rather than waiting for the next product edit.

4. Deferred for Production

The ADRs above evolved five seams under pressure. Three concerns at the operational layer were deliberately deferred and would be addressed before any real-traffic rollout. None affects the QA response measures the demo verifies; each becomes a real concern at the volumes and uptime expectations of a production deployment.

First: outbox cleanup. The relay marks rows as published by updating `PublishedAt` but never deletes them. For the demo this is harmless because volume is low, but in production the table would grow unbounded and degrade the relay poll over time. The fix is a periodic background job that deletes rows past a retention window. Seven days is a defensible default for audit purposes. The job can be a Hangfire scheduled task or a database-level cron.

Second: distributed tracing. There is no distributed tracing, only Prometheus metrics. The end-to-end path of an order from the storefront through the outbox, broker, and CRM consumer is observable in aggregate through queue depth, publish rate, and consumer lag, but it cannot be reconstructed as a single correlated trace per order. Adding OpenTelemetry trace propagation across the in-process event bus, the relay loop, the broker using the W3C TraceContext header, and the consumer would give per-order observability. The cost is a tracing backend such as Jaeger or Tempo, plus the discipline of propagating context across every async boundary.

Third: single-host, single-instance deployment. Every container runs as a single instance on one Docker host: the two BU storefronts `nop_bu1` and `nop_bu2`, and the four group-level services Keycloak, RabbitMQ, Meilisearch, and EspoCRM. The demo's goal is to prove per-BU resilience and the seam contracts, not the operational maturity of the deployment. The architecture supports scaling each of them out: BU storefronts on separate VMs or Kubernetes namespaces, Keycloak as an HA pair sharing a database, RabbitMQ as a quorum-queue cluster, Meilisearch with a read replica, and EspoCRM behind a load balancer with shared MariaDB. None of this requires code changes, and it would be the first hardening step before any real-traffic rollout.

5. Setup and Run Instructions

The runnable system requires Docker Engine 24 or later with Compose v2 and bash. The host needs the ports 8000, 8080-8083, 9001-9003, 9101-9102, 7700, 5672, 15672, 15692, 3000, and 9090 free. The stack is brought up with one compose command and one idempotent installer:

```
git clone <repo-url>
cd AS_Group
docker compose -f docker-compose.yml -f docker-compose.observability.yml up --build -d
./scripts/install-nop.sh
```

The installer seeds both BUs, installs the four plugins, wires the Keycloak, Meilisearch, ERP, RabbitMQ settings, and is safe to re-run on an already-installed stack.

Six smoke-test scripts produce PASS or FAIL per ADR:

- `test-isolation.sh` (ADR-001);
- `test-sso.sh` (ADR-002);
- `test-outbox.sh` and `test-durability.sh` (ADR-003);
- `test-erp-failure.sh` followed by `test-erp-recovery.sh` (ADR-004);
- `test-search.sh` (ADR-005).

Three demo scripts drive failure-mode demos end to end with stamped banners showing when each failure phase is hot:

- `demo-search-fallback.sh` (ADR-005): starts k6 search-load and pauses Meilisearch mid-run;
- `demo-breaker.sh` (ADR-004): starts k6 product-detail-load and breaks the BU1 ERP mid-run;
- `demo-outbox.sh` (ADR-003): pauses the EspoCRM consumer and injects 100 rows, processed in two relay batches of 50.

`demo-cross-bu-crm.sh` (ADR-003) is a positive-path demo rather than a failure injection: it places one order in each BU for the same customer and shows both folded onto a single EspoCRM contact, proving the shared-customer view from Scenario A.

Open `http://localhost:3000/d/northstar-adr-evidence` during each run, set the time picker to "Last 5 min" with 5-second refresh, and watch the relevant row when the script prints the "outage hold" or "watching drain" phase. Trap handlers undo the disruptive action on Ctrl+C.

Table 1: Service URLs and credentials.

Service	**URL**	**Credentials**
Group Portal	`http://localhost:8000`	-
BU1 storefront (HomeStyle)	`http://localhost:8081`	-
BU2 storefront (WorkSpace)	`http://localhost:8082`	-
Keycloak admin	`http://localhost:8080`	admin / admin
RabbitMQ management	`http://localhost:15672`	northstar / northstar
Meilisearch	`http://localhost:7700`	master key in compose
EspoCRM	`http://localhost:8083`	admin / admin
ERP stub BU1	`http://localhost:9001/health`	-
ERP stub BU2	`http://localhost:9002/health`	-
Prometheus	`http://localhost:9090`	-
Grafana	`http://localhost:3000`	admin / admin